1) Recap

2) Feed Forward Neural Networks (FFNN)

2.1) Recap On Perceptron

The perceptron is a **binary classification algorithm**, that is:

$$f : \mathbb{R}^d \to \{-1, +1\}, \quad x \to t$$

It divides the space into two regions divided by a hyperplane. These regions are called **halfspaces**:

$$H = \left\{x \to sign(w^T x + b) : w \in \mathbb{R}^d, b \in \mathbb{R}\right\}$$

The decision function f is the sign: $y = f(x) = sign(w^T x + b)$

Hyperplane

The hyperplane is the points that satisfy $w^T x = K$. The hyperplane has direction given by w and the distance from the origin is $w^T x = \|w\|\|x\| \cos(\theta)$.

Now add the bias b. The hyperplane is the set of points that satisfy

$$w^T x + b = 0$$

and we define v as the vector from the origin to the closest point of the hyperplane (perpendicular)

$$v = d\frac{w}{\|w\|} = d \cdot u_w$$

The distance can be found by substituting $x = v$:

$$w^T x + b \overset{x=v}{=} w^T d\frac{w}{\|w\|} + b \overset{w^T w = \|w\|^2}{=} d\|w\| + b = 0 \to d = -\frac{b}{\|w\|}$$

From here we get that: **w gives the orientation and b the distance from origin**

for convenience the bias is added as a weight:

$$w_d = \begin{bmatrix} w_1 \\ \dots \\ w_d \end{bmatrix}, \; x_d = \begin{bmatrix} x_1 \\ \dots \\ x_d \end{bmatrix} \to w_{d+1} = \begin{bmatrix} w_1 \\ \dots \\ w_d \\ b \end{bmatrix}, \; x_{d+1} = \begin{bmatrix} x_1 \\ \dots \\ x_d \\ 1 \end{bmatrix}$$

so that $y = sign(w_{d+1}^T x_{d+1}) = sign(w_d^T x_d + b)$.

Training

Given a dataset of labeled points:

$$\mathcal{D} = \{(x_1, t_1), \dots, x_N, t_N\} \quad t_n \in \{-1, +1\}$$

start with $w^{(0)} = \vec{0}$

A correct classification is when $(w^{(i)})^T x_n t_n > 0$.

We must minimize the loss function:

$$\min \mathcal{L}(w, \mathcal{D}) = \min \sum_{n \in \mathcal{D}} l(w, x_n) = \min \sum_{n \in \mathcal{D}} \max(0, -w^T x_n t_n)$$

Weights are updated when a sample is incorrectly classified:

$$w^{(i+1)} = w^{(i)} + x_n t_n$$

At each iteration we are always improving the loss function

Proof:

$$(w^{(i+1)})^T x_n t_n = (w^{(i)} + x_n t_n)^T x_n t_n = (w^{(i)})^T x_n t_n + \|x_n\|^2 > (w^{(i)})^T x_n t_n$$

□

Limitations

This only works for linearly separable data.

A possible solution is feature mapping:

$$\phi(x), \phi : \mathbb{R}^d \to \mathbb{R}^m$$

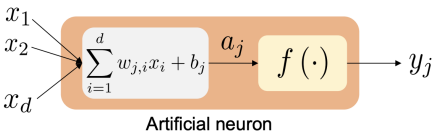
but a good ϕ is hard to find. **Neural networks learn the feature mapping**
There are infinitely many feature mappings, we base them on two insights:

1. ϕ is built as a composition of multiple simpler functions ($\phi = f\ o\ g\ o\ h\ o \ldots$)
2. ϕ has a multi-layered stucture with increasing abstraction

2.2) FFNN

NN are able to learn the feature mapping. Each smaller function (neuron) is based on the perceptron.
Instead of a sign function a differentiable activation function is used.

$$y_j = f(a_j) = f\left(\sum_{i=1}^{d}\right) w_{j,i} x_i + b_j$$



Representation

2.3) Single Layer FFNN

A layer of a FFNN is a **stack of multiple neurons**.
The input vector is fed to a hidden layer (neurons) that have:
-An affine transformation

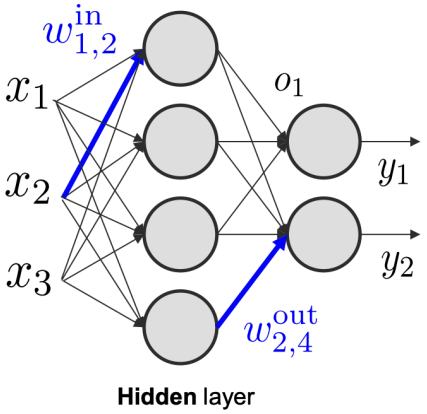
$$a = W^{in} z + b^{in}$$

-A non-linear activation function

$$o = f(a)$$

The output is therefore:

$$y = g(W^{out} o + b^{out})$$



Representation

Theorem 1 (Universal Approximation Capability)**.**

Single-layer FFNN can approximate any function with arbitrary accuracy given a suitable activation function

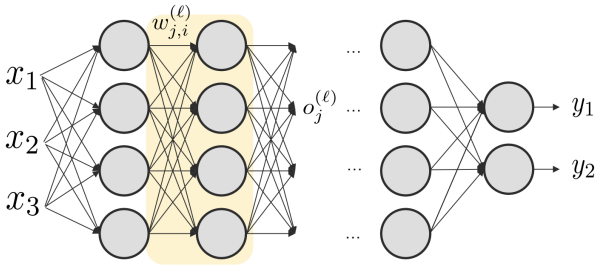
The number of hidden neurons required is exponential in the dimension of the input space d. This means single-layer FFNNs are of limited use in solving practical problems. Multiple layers can be used to solve this problem.

2.4) Multi-Layered FFNN

Notation

Start with some notation:

- $w_{i,j}^{(l)}$: weights that connects neuron i of layer $l-1$ with neuron j of layer l
- $W^{(l)}, b^{(l)}$: parameters of layer l
- $a_j^{(l)}, o_j^{(l)}$: activation and output of neuron j of layer l
- $\mathcal{W}$: set of FFNN parameters (weights and biases)
- $\mathcal{L}(\mathcal{W}, x)$: loss function of FFNN on input sample x
- $\mathcal{D} = \{(x_1, t_1), \ldots, (x_N, t_N)\}$: labeled training data



Repreentation

recall: $w_{\mathbf{j,i}}$ means direction $i \to j$.

Input-Output Relation: Forward Pass

Given an input x, the output is obtained with the forward pass:

- feed x to the first layer: $a^{(1)} = W^{(1)}x + b^{(1)},\ o^{(1)} = f(a^{(1)})$
- Use $o^{(1)}$ as input for the second layer
- Repeat for all layers
- Get output vector as: $a^{(L)} = W^{(L)}o^{(L-1)} + b^{(L)},\ y = g(a^{(L)})$

$a^{(l)}, o^{(l)}\ \forall l \in \{1, \ldots, L\}$ are stored for the back propagation.

Loss Function

The loss function represents the optimization objective of the NN training. It should reflect the learning task.

Typically:

- Negative log-likelihood (NLL) of parameters
- Assume **i.i.d** training samples $\to$ decompose NLL as sum of terms

$$\mathcal{L}(\mathcal{D}, \mathcal{W}) = -\log p(\{t_n\}_{n=1}^N | \{x_n\}_{n=1}^N; \mathcal{W}) = -\sum_{n=1}^N \log p(t_n | x_n; \mathcal{W})$$

Regression (Continuous Labels and Output)

Output Function

for now g is the identity function, therefore $y_n = a_n^{(L)}$
It allows to span the whole target space $\mathbb{R}^n$

Error Function: Mean Squared Error (MSE)

Suppose that label is given by the output + some gaussian noise: $t_n = y_n + \text{noise}$
Then the **per sample likelihood** can be written as a gaussian:

$$\underbrace{p(t_n|x_n;\mathcal{W})}_{\text{objective}} = \mathcal{N}(t_n; y_n, \Sigma)$$

where we suppose y_n is the mean and Σ the covariance.

By setting $\Sigma = I$ so to focus on predicting the mean we can find the following loss function:

$$\begin{aligned} -\log(\mathcal{N}(t_n; y_n, \Sigma)) &= \frac{1}{2}\left[\log\det\Sigma + d\log 2\pi + (y_n - t_n)^T\Sigma^{-1}(y_n - t_n)\right] \\ &\overset{\Sigma=I}{=} \frac{1}{2}\left[d\log 2\pi + (y_n - t_n)^T(y_n - t_n)\right] \\ &\downarrow \\ \mathcal{L}(\mathcal{D}, \mathcal{W}) &\propto \frac{1}{2}\sum_{n=1}^{N}\|y_n - t_n\|^2 \end{aligned}$$

Classification (Discrete Labels, Continuous Output)

Since this is a classification problem, consider the binary classification case:

- 1 output neuron with $t_n \in \{0, 1\}$
- The two classes are called $\mathcal{C}_0, \mathcal{C}_1$

Output Function

Usually the sigmoid function is used:

$$y_n = g(a_n^{(L)}) = \sigma(a_n^{(L)}) = \frac{1}{1 + \exp\left\{-a_n^{(L)}\right\}}$$

This maps $\mathbb{R} \to [0, 1]$
This can be interpreted as the probability of the output being 1 given the input: $p(\mathcal{C}_1|x_n)$
Or alternatively the *complementary class:* $1 - y_n = p(\mathcal{C}_0|x_n)$.

Error Function: Cross-Entropy

Recalling the complementary class, the likelihood can be expressed as:

$$p(t_n|x_n;\mathcal{W}) = p(\mathcal{C}_1|x_n)^{t_n}p(\mathcal{C}_0|x_n)^{1-t_n}$$

and the NLL becomes:

$$\begin{aligned} -\log p(t_n|y_n, \mathcal{W}) &= -t_n \log p(\mathcal{C}_1|x_n) - (1 - t_n)\log p(\mathcal{C}_0|x_n) \\ &= -t_n \log y_n - (1 - t_n)\log(1 - y_n) \\ \mathcal{L}(\mathcal{D}, \mathcal{W}) &= -\sum_{n=1}^{N}\log p(t_n|x_n;\mathcal{W}) \\ &= -\sum_{n=1}^{N} t_n\log(y_n) + (1 - t_n)\log(1 - y_n) \end{aligned}$$

Generalization to multi-class

This can be generalized to multi-class classification problems.

- Let there be Q labels
- Labels are expressed in one-of-Q representation (one-hot encoding)
- Use Q output neurons, each one dedicated to one of the labels

if x_n belongs to a class q, then set $t_{n,q} = 1, t_{n,i} = 0 \forall i \neq q$.
this can be seen as $y_{n,q} = p(t_{n,q} = 1|x_n)$

The error function is the **categorical cross entropy**

$$\mathcal{L}(\mathcal{D}, \mathcal{W}) = -\sum_{n \in N} \sum_{q \in Q} t_{n,q} \log(y_{n,q})$$

Softmax Output

In multi-class, the **output should be a discrete probability distribution overt the Q classes**. Therefore:

- $y_{n,q} \in [0, 1], \ \forall q$
- $\sum_{q \in Q} y_{n,q} = 1$

The most common choice is the **softmax function**

$$y_{n,q} = \frac{\exp\left\{a_q^{(L)}\right\}}{\sum_{i \in Q} \exp\left\{a_i^{(L)}\right\}}$$

2.5) FFNN Training